CS336 Assignment 1 要求中文译文

从零构建 Transformer 语言模型

依据本地英文讲义：CS336 Assignment 1 (basics): Building a Transformer LM，2026 春季，版本 26.0.3。

翻译范围：所有题目、实现要求、实验要求和提交要求，以及完成这些任务所需的接口、公式与配置。背景介绍、完整推导和大部分演示示例未逐段翻译。本译文不包含题目解答。文中的“原文第 X 页”指英文 PDF 的页码。

1 作业总要求

1.1 需要实现和运行的内容

本作业要求从零实现训练标准 Transformer 语言模型所需的全部组件，并实际训练模型。

需要实现：

- 字节级字节对编码分词器（Byte-Pair Encoding，BPE），见第 2 节。
- Transformer 语言模型，见第 3 节。
- 交叉熵损失函数与 AdamW 优化器，见第 4 节。
- 训练循环，包括模型状态与优化器状态的保存和加载，见第 5 节。

需要运行：

1. 在 TinyStories 数据集上训练 BPE 分词器。
2. 用训练好的分词器编码数据集，把文本转换成整数 token ID 序列。
3. 在 TinyStories 上训练 Transformer 语言模型。
4. 使用训练好的模型生成文本，并评估困惑度（perplexity）。
5. 在 OpenWebText 上训练模型，并向排行榜提交成绩；具体指标与限制见原文第 7.5 节，对应本译文的 leaderboard 题。

1.2 允许使用的工具与库

各个组件都应从零构建。除下列例外，不得使用 torch.nn、torch.nn.functional 或 torch.optim 中的任何定义：

- torch.nn.Parameter。
- torch.nn 中的容器类，例如 Module、ModuleList、Sequential 等。
- torch.optim.Optimizer 基类。

可以使用 PyTorch 中其他定义。如果不确定某个函数或类是否允许使用，可在课程 Slack 上询问；判断原则是使用它是否违背“从零实现”的作业目的。

原文第 3.3 节另有明确的具体要求：线性层和嵌入层的截断正态初始化应使用 torch.nn.init.trunc_normal_。这一初始化函数属于后文明确允许使用的例外。

1.3 原讲义中的 AI 工具规定

允许使用 AI 回答高层次的概念问题，或查询底层编程文档，例如函数签名和库 API。不允许使用 AI 实现任何作业部分，这包括编码代理和 AI 自动补全。使用 AI 代理时，应让它遵循仓库中的 AGENTS.md；使用聊天机器人时，也应提供相应指令。

课程强烈建议完成作业时关闭 IDE 中的 AI 自动补全。普通的非 AI 自动补全，例如补全函数名，可以使用。

1.4 代码组织与测试

- cs336_basics/*：在这里编写自己的实现。起始框架很少，整体组织方式可自行决定。
- tests/adapters.py：把自己的实现接到课程测试接口上。例如，run_scaled_dot_product_attention 应调用自己的注意力实现。这里应只放连接接口的代码，不应包含实质性实现逻辑。
- tests/test_*.py：必须通过的测试，会调用 adapters.py 中的接口。不要修改测试文件。

课程代码仓库：https://github.com/stanford-cs336/assignment1-basics

使用 git clone 获取仓库；课程通知更新时，用 git pull 获取最新内容。

本地 README 的环境设置说明：项目使用 uv 管理环境；可通过 uv run <python_file_path> 运行 Python 文件，uv 会按需解析依赖并启用环境。运行全部单元测试的命令为 uv run pytest。尚未连接实现时，测试抛出 NotImplementedError 是预期现象。

1.5 数据集与低资源说明

本作业使用已经预处理的 TinyStories 和 OpenWebText；两者均以大型纯文本文件提供。正式选课学生应参照课程计算资源指南获取数据；自学者可使用本地 README.md 中的下载命令。

原讲义中的低资源提示适用于 GPU 较少或没有 GPU 的情况，包括缩小数据集、缩小模型和使用 CPU 或 Apple Silicon 的 MPS 后端。第 7 节保留了具体的替代训练量与验证损失要求。讲义中记录的运行时间是课程参考实现的测量结果，并非对任意设备的速度保证。

1.6 提交方式

运行 make_submission.sh 生成提交用 ZIP。如果仓库中有不应打包的大型数据文件或检查点，需把它们加入脚本的排除列表。

向 Gradescope 提交：

- writeup.pdf：回答所有书面题，请使用排版后的文档。
- code.zip：包含自己编写的全部代码。

排行榜通过向下列仓库提交 Pull Request 参与，详细步骤见其 README.md：

https://github.com/stanford-cs336/assignment1-basics-leaderboard

以上对应原文第 1 至 3 页。本地 README 标题仍写 Spring 2025，但本译文的题目与要求以实际 PDF 的 Spring 2026、版本 26.0.3 为准。

2 字节对编码分词器 BPE Tokenizer

本部分要求训练并实现一个字节级 BPE 分词器：将 Unicode 字符串表示为 UTF-8 字节序列，在字节序列上训练 BPE；随后把文本编码成用于语言建模的整数 token ID 序列。

unicode1 理解 Unicode（1 分）

原文第 3-4 页。

(a) `chr(0)` 返回哪个 Unicode 字符？

提交内容：一句话回答。

(b) 这个字符的字符串表示（`__repr__()`）与打印出来的表示有什么不同？

提交内容：一句话回答。

(c) 当这个字符出现在文本中时，会发生什么？可以在 Python 解释器中试验下面的原题示例，观察结果是否符合预期。

```
>>> chr(0)
>>> print(chr(0))
>>> "this is a test" + chr(0) + "string"
>>> print("this is a test" + chr(0) + "string")
```

提交内容：一句话回答。

unicode2 Unicode 编码（3 分）

原文第 4-5 页。

(a) 为什么训练分词器时，可能更适合使用 UTF-8 编码的字节，而不是 UTF-16 或 UTF-32？可以比较不同输入字符串在这些编码下的输出。

提交内容：一到两句话回答。

(b) 下面这个错误函数试图把 UTF-8 字节串解码成 Unicode 字符串。为什么它不正确？给出一个会产生错误结果的输入字节串。

```
def decode_utf8_bytes_to_str_wrong(bytestring: bytes):
    return "".join([bytes([b]).decode("utf-8") for b in bytestring])
>>> decode_utf8_bytes_to_str_wrong("hello".encode("utf-8"))
'hello'
```

提交内容：一个使 `decode_utf8_bytes_to_str_wrong` 产生错误输出的输入字节串，并用一句话解释函数为何错误。

(c) 给出一个无法解码为任何 Unicode 字符的双字节序列。

提交内容：一个例子，并用一句话解释。

BPE 训练必须遵守的规则

原文第 5-9 页，供后面的实现题使用。

- 初始字节词表包含全部 256 种字节值。词表把字节串 token 与整数 ID 一一对应。
- 先对文本预分词，再把每个预分词片段表示为 UTF-8 字节序列。使用以下 GPT-2 风格的正则表达式；它需要 Python 的 `regex` 包。

```
PAT = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
```

- 实际统计预分词片段频次时，应使用 `re.finditer`，避免额外保存全部预分词结果。

- 每轮选择出现频次最高的相邻 token 对，将其所有出现位置合并成一个新 token，并加入词表。合并不能跨越预分词片段的边界。
- 如果多个 token 对的频次相同，确定性地选择字典序最大的那一对。例如，原文在（“A”, “B”）、（“A”, “C”）、（“B”, “ZZ”）、（“BA”, “A”）并列时选择（“BA”, “A”）。
- 本作业的字节级 BPE 不额外添加词尾 token，因为空格、标点等字节已在词表中。
- 特殊 token（如 `<|endoftext|>`）必须加入词表，并在编码时始终保留为一个 token，即一个整数 ID。
- 训练时，在运行正则预分词前，先按所有特殊 token 切开语料，并移除特殊 token 本身。分别处理切分后的文本，不能在移除特殊 token 后把两侧文本接起来再预分词。特殊 token 构成不可跨越的硬边界，但本身不参与合并频次统计。`test_train_bpe_special_tokens` 会检查这一点。
- 原文说明可使用 `re.split` 和 `"|".join(special_tokens)` 切分，但必须注意用 `re.escape` 转义特殊 token 中可能存在的 `|` 等正则字符。

原文给出的性能建议：可用 `multiprocessing` 并行处理预分词，让每个数据块的边界位于特殊 token 的开头。允许直接使用仓库中的 `cs336_basics/pretokenization_example.py` 来确定切块边界。本作业中可以假定语料含有 `<|endoftext|>`，不必处理一个极大语料完全没有该标记的特殊情况。

合并阶段可索引所有相邻对的频次，并在每轮合并后增量更新受影响的频次，避免重新遍历全部字节对。原文指出，BPE 的合并部分不能在 Python 中并行化。建议用 `cProfile` 或 `py-spy` 定位瓶颈；可以先在较小的调试数据集上开发，例如 TinyStories 验证集（约 2.2 万篇文档，而完整训练集约 212 万篇）。调试规模应足以暴露完整配置中的主要瓶颈。

train_bpe 训练 BPE 分词器（15 分）

原文第 9 页。

提交内容：编写一个函数，接受输入文本文件的路径，训练一个字节级 BPE 分词器。函数至少应接受以下输入参数。

- `input_path: str`：包含 BPE 训练数据的文本文件路径。
- `vocab_size: int`：正整数，指定最终词表大小的上限；该大小包括初始字节词表、合并产生的词表项以及全部特殊 token。
- `special_tokens: list[str]`：要加入词表的特殊字符串列表。训练时，把它们当作阻止跨边界合并的硬边界，但不把它们计入合并统计。

函数应返回以下结果。

- `vocab: dict[int, bytes]`：分词器词表，从整数 token ID 映射到 token 的字节串。
- `merges: list[tuple[bytes, bytes]]`：训练产生的 BPE 合并列表。每项是字节串二元组 `(<token1>, <token2>)`，表示将这两个 token 合并。列表必须按合并规则产生的先后顺序排列。

先接好测试适配器 `adapters.run_train_bpe`，然后运行：

```
uv run pytest tests/test_train_bpe.py
```

实现应通过全部测试。

可选：可以用 C++（如 `cppyy` 或 `nanobind`）或 Rust（如 PyO3）实现训练方法的关键部分，但这可能很耗时。如果这样做，要明确哪些操作会复制 Python 内存、哪些直接读取 Python 内存；提供构建说明，或者保证仅靠 `pyproject.toml` 即可完成构建。大部分正则引擎不完整支持所需的 GPT-2 正则表达式，部分支持的引擎也很慢。原文称已验证 Oniguruma 支持负向前瞻且速度较好，但 Python 的 `regex` 包甚至更快。

train_bpe_tinystories 在 TinyStories 上训练 BPE（2 分）

原文第 9-10 页。

(a) 在 TinyStories 数据集上训练字节级 BPE 分词器，最大词表大小设为 10,000。务必把 TinyStories 的 `<|endoftext|>` 特殊 token 加入词表。把得到的词表和合并列表序列化到磁盘，供后续检查。训练耗时和内存占用分别是多少？词表中最长的 token 是什么？它合理吗？

资源要求：不使用 GPU，时间不超过 30 分钟，内存不超过 30 GB。

原文提示：通过并行预分词，并利用下面两个事实，应该能够把 BPE 训练时间降到 2 分钟以内：
`<|endoftext|>` 分隔数据文件中的各篇文档；在应用 BPE 合并前，它会作为特殊情况单独处理。

提交内容：一到两句话回答。

(b) 对代码做性能分析。分词器训练过程中哪一部分最耗时？

提交内容：一到两句话回答。

train_bpe_expts_owt 在 OpenWebText 上训练 BPE（2 分）

原文第 10 页。

(a) 在 OpenWebText 数据集上训练字节级 BPE 分词器，最大词表大小设为 32,000。把得到的词表和合并列表序列化到磁盘，供后续检查。词表中最长的 token 是什么？它合理吗？

资源要求：不使用 GPU，时间不超过 12 小时，内存不超过 100 GB。

提交内容：一到两句话回答。

(b) 比较在 TinyStories 和 OpenWebText 上训练得到的两个分词器，指出它们的相同点和不同点。

提交内容：一到两句话回答。

编码与解码的实现要求

原文第 10-11 页。

- 编码时，使用与训练时相同的预分词方式，把每个预分词片段表示为 UTF-8 字节序列；各片段独立处理，不能跨片段合并。
- 按训练时合并规则产生的先后顺序应用这些规则。
- 正确处理构造分词器时提供的用户自定义特殊 token。
- 对无法全部放进内存的大文件或数据流，应分块处理，使内存复杂度不随文本总长度线性增长。切块边界不能截断 token，否则会与一次性对完整文本分词的结果不同。
- 解码时，先查询各 token ID 对应的字节串，把它们连接起来，再解码为 Unicode 字符串。任意输入 ID 序列不一定能组成合法的 UTF-8 字符串；对其中无效的字节，应使用 Unicode 替换字符 U+FFFD。原文提示 `bytes.decode` 的 `errors='replace'` 可完成这一处理。

tokenizer 实现分词器（15 分）

原文第 11-12 页。

提交内容：实现一个 `Tokenizer` 类。给定词表和合并列表，它能够把文本编码为整数 ID，也能够把整数 ID 解码为文本。它还应支持用户提供的特殊 token；若这些 token 尚不在词表中，就把它们追加到词表。

原文建议使用以下接口。

```
def __init__(self, vocab, merges, special_tokens=None)
```

根据给定词表、合并列表和可选的特殊 token 列表构造分词器。接受的参数类型如下：

- `vocab: dict[int, bytes]`

- `merges: list[tuple[bytes, bytes]]`
- `special_tokens: list[str] | None = None`

`def from_files(cls, vocab_filepath, merges_filepath, special_tokens=None)`

类方法：从序列化的词表和合并列表，以及可选的特殊 token 列表，构造并返回一个 `Tokenizer`。文件格式应与自己 BPE 训练代码的输出格式相同。接受以下参数：

- `vocab_filepath: str`
- `merges_filepath: str`
- `special_tokens: list[str] | None = None`

`def encode(self, text: str) -> list[int]`

把输入文本编码成 token ID 序列。

`def encode_iterable(self, iterable: Iterable[str]) -> Iterator[int]`

接受一个可迭代的字符串对象（例如 Python 文件句柄），返回一个惰性产生 token ID 的生成器。此接口用于以较低内存占用处理无法整体加载到内存的大文件。

`def decode(self, ids: list[int]) -> str`

把 token ID 序列解码成文本。

先接好测试适配器 `adapters.get_tokenizer`，然后运行：

```
uv run pytest tests/test_tokenizer.py
```

实现应通过全部测试。

tokenizer_experiments 分词器实验（4 分）

原文第 12-13 页。

(a) 分别从 TinyStories 和 OpenWebText 中抽取 10 篇文档。使用之前训练得到的相应分词器（TinyStories 词表大小为 10K，OpenWebText 为 32K），把抽样文档编码成整数 ID。每个分词器的压缩率是多少？这里用每个 token 对应的字节数（bytes/token）表示压缩率。

提交内容：一到两句话回答。

(b) 如果用 TinyStories 分词器处理 OpenWebText 抽样文档，会发生什么？比较压缩率，和 / 或对观察到的现象作定性描述。

提交内容：一到两句话回答。

(c) 估计分词器的吞吐量，例如每秒处理的字节数。用它对 Pile 数据集（825 GB 文本）分词，需要多久？

提交内容：一到两句话回答。

(d) 使用 TinyStories 和 OpenWebText 分词器，把各自对应的训练集与开发集编码成整数 token ID 序列，之后会用来训练语言模型。建议将 token ID 序列序列化为数据类型为 `uint16` 的 NumPy 数组。为什么 `uint16` 是合适的选择？

提交内容：一到两句话回答。

3 Transformer 语言模型架构

本节要求从头实现 Transformer 语言模型。输入是形状为 (batch_size, sequence_length) 的整数 token ID 张量；模型为序列中的每个位置预测下一个 token。完整模型输出形状为 (batch_size, sequence_length, vocab_size) 的未归一化分数，即 logits。训练时用这些分数计算交叉熵；生成时使用最后一个位置的预测分布。

整体结构为：token 嵌入层 → num_layers 个 pre-norm Transformer 块 → 最终归一化层 → 线性输出层（LM head）。每个 Transformer 块的输入、输出形状均为 (batch_size, sequence_length, d_model)。

通用实现规范（原文第 **13-17** 页）

- 批次、序列位置和注意力头都可能成为批处理维度。函数应能处理前置的额外批处理维度，并尽可能使用高效的批量张量运算。
- 讲义强烈建议学习并使用 einsum 记号。可使用 torch.einsum、einops 或 einx；课程环境已经安装后两者。它们是建议使用的工具，并非每道题都强制要求。
- 讲义的数学公式主要采用列向量记法：y = W x，其中权重矩阵形状为 (d_out, d_in)。若以 PyTorch 常见的末维存特征、前置维存批次的张量布局进行普通矩阵乘法，对应写法是 y = x W^T。权重仍应按 (d_out, d_in) 存储。
- 线性层不使用偏置项。
- 线性层权重初始化：均值为 0、方差为 2 / (d_in + d_out) 的正态分布，截断范围为 [-3σ, 3σ]，其中 σ 是标准差。
- 嵌入层权重初始化：均值为 0、方差为 1 的正态分布，截断范围为 [-3, 3]。
- RMSNorm 的可学习增益参数初始化为全 1。
- 讲义在此明确要求使用 torch.nn.init.trunc_normal_ 初始化截断正态权重。

题目 **linear**：实现线性模块（**1** 分，原文第 **18** 页）

提交内容：实现继承 torch.nn.Module 的 Linear 类，完成线性变换。接口应参照 PyTorch 内置的 nn.Linear，但不要提供偏置参数或偏置选项。建议接口如下。

__init__(self, in_features, out_features, device=None, dtype=None)：构造线性变换模块。

- in_features: int：输入的最后一维大小。
- out_features: int：输出的最后一维大小。
- device: torch.device | None = None：参数存放的设备。
- dtype: torch.dtype | None = None：参数的数据类型。

forward(self, x: torch.Tensor) -> torch.Tensor：对输入执行线性变换。

务必做到：继承 nn.Module；调用父类构造函数；以 W 而不是 W^T 的形状构造并保存权重，将其包装为 nn.Parameter；不要使用 nn.Linear 或 nn.functional.linear。按上面的初始化设置，使用 torch.nn.init.trunc_normal_ 初始化权重。

测试：实现适配器 adapters.run_linear。适配器应将测试提供的权重加载到你的 Linear 模块中，可使用 Module.load_state_dict。随后运行 uv run pytest -k test_linear。

题目 **embedding**：实现嵌入模块（**1** 分，原文第 **18-19** 页）

提交内容：实现继承 torch.nn.Module 的 Embedding 类，完成嵌入查表。接口应参照 PyTorch 内置的 nn.Embedding。建议接口如下。

__init__(self, num_embeddings, embedding_dim, device=None, dtype=None)：构造嵌入模块。

- num_embeddings: int：词表大小。
- embedding_dim: int：嵌入向量维度，即 d_model。
- device: torch.device | None = None：参数存放的设备。
- dtype: torch.dtype | None = None：参数的数据类型。

forward(self, token_ids: torch.Tensor) -> torch.Tensor：查找给定 token ID 对应的嵌入向量。

嵌入矩阵的形状应为 (vocab_size, d_model)。用形状为 (batch_size, sequence_length) 的 token ID torch.LongTensor 索引该矩阵，得到形状为 (batch_size, sequence_length, d_model) 的输出。

务必做到：继承 nn.Module；调用父类构造函数；将嵌入矩阵初始化为 nn.Parameter；存储矩阵时将 d_model 放在最后一维；不要使用 nn.Embedding 或 nn.functional.embedding。按前面的初始化设置，使用 torch.nn.init.trunc_normal_ 初始化权重。

测试：实现 adapters.run_embedding，运行 uv run pytest -k test_embedding。

题目 **rmsnorm**：均方根层归一化（**1** 分，原文第 **19-20** 页）

本作业使用 RMSNorm。对最后一维上的激活向量 a，其定义为：

RMS(a) = sqrt((1 / d_model) × sum_i(a_i^2) + ε)

RMSNorm(a)_i = (a_i / RMS(a)) × g_i

其中 g 是包含 d_model 个元素的可学习增益参数；ε 通常取 1e-5。

提交内容：将 RMSNorm 实现为 torch.nn.Module。建议接口如下。

__init__(self, d_model: int, eps: float = 1e-5, device=None, dtype=None)：构造 RMSNorm 模块。

- d_model: int：模型的隐藏维度。
- eps: float = 1e-5：用于数值稳定的 epsilon。
- device: torch.device | None = None：参数存放的设备。
- dtype: torch.dtype | None = None：参数的数据类型。

forward(self, x: torch.Tensor) -> torch.Tensor：处理形状为 (batch_size, sequence_length, d_model) 的输入，返回相同形状的张量。

数值要求：归一化前将输入转换为 torch.float32，避免对输入求平方时溢出；返回前将结果转回输入原本的数据类型。

测试：实现 adapters.run_rmsnorm，运行 uv run pytest -k test_rmsnorm。

题目 **positionwise_feedforward**：实现逐位置前馈网络（**2** 分，原文第 **21-22** 页）

提交内容：实现由 SiLU 激活函数与门控线性单元 GLU 组成的 SwiGLU 前馈网络。各线性层均不使用偏置。原文给出的定义为：

SiLU(x) = x × sigmoid(x) = x / (1 + exp(-x))

FFN(x) = W2 (SiLU(W1 x) ⊙ W3 x)

其中 ⊙ 表示逐元素乘法；W1、W3 的形状为 (d_ff, d_model)，W2 的形状为 (d_model, d_ff)。

实现时将 d_ff 设为约 (8/3) × d_model，并保证前馈网络内部维度是 64 的倍数，以充分利用硬件。可将该值取整到附近的 64 的倍数。

允许使用：为保证数值稳定，这里可以使用 torch.sigmoid。

测试：实现 adapters.run_swiglu，运行 uv run pytest -k test_swiglu。

题目 **rope**：实现旋转位置嵌入 **RoPE**（**2** 分，原文第 **22-23** 页）

RoPE 将查询向量和键向量中相邻的两个分量视为二维向量并旋转。对位置 i、第 k 对分量，旋转角为：

θ_(i,k) = i / θ^((2k-2)/d)，其中 k = 1, ..., d/2。

对应的二维旋转矩阵为 [[cos θ, -sin θ], [sin θ, cos θ]]。整个变换由这些二维旋转组成的块对角矩阵表示。查询向量按自己的位置旋转，键向量也按自己的位置执行相同形式的旋转。

这个模块没有可学习参数。实现应利用矩阵结构提高效率，而不是必须显式构造完整的 d × d 矩阵。若希望优化，可让所有层引用同一个 RoPE 模块，并在初始化时预先计算二维的正弦、余弦缓冲区；应通过 self.register_buffer(persistent=False) 注册，而不是将其设为 nn.Parameter。这些数值可在不同层和不同批次间复用。

提交内容：实现 RotaryPositionalEmbedding 类，对输入张量应用 RoPE。建议接口如下。

__init__(self, theta: float, d_k: int, max_seq_len: int, device=None)：构造 RoPE 模块，按需创建缓冲区。

- theta: float：RoPE 的常数 θ。
- d_k: int：查询和键向量的维度。
- max_seq_len: int：可能输入的最大序列长度。
- device: torch.device | None = None：缓冲区存放的设备。

forward(self, x: torch.Tensor, token_positions: torch.Tensor) -> torch.Tensor：处理形状为 (..., seq_len, d_k) 的输入，返回相同形状的张量。应支持 x 包含任意数量的批处理维度。token_positions 的形状为 (..., seq_len)，指定 x 沿序列维度的各个 token 位置。

应使用这些 token 位置，沿序列维度索引或切片正弦、余弦张量；这些张量可以预先计算。

测试：实现 adapters.run_rope，确保通过 uv run pytest -k test_rope。

题目 **softmax**：实现 **softmax**（**1** 分，原文第 **23-24** 页）

提交内容：编写函数，对输入张量指定的第 i 维执行 softmax。函数接收两个参数：输入张量和维度 i。输出形状必须与输入相同，但沿第 i 维的值应构成归一化的概率分布。

原文定义：softmax(v)_i = exp(v_i) / sum_j(exp(v_j))。

数值要求：先从该维度的所有元素中减去该维度的最大值，再计算 softmax，以避免数值稳定性问题。

测试：实现 adapters.run_softmax，确保通过 uv run pytest -k test_softmax_matches_pytorch。

题目 **scaled_dot_product_attention**：实现缩放点积注意力（**5** 分，原文第 **24-25** 页）

原文定义：Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V。这里的 Q、K、V 是此运算的输入，不是可学习参数。一般定义中，它们的形状分别为 (n, d_k)、(m, d_k)、(m, d_v)。

提交内容：实现缩放点积注意力函数。实现应支持形状为 (batch_size, ..., seq_len, d_k) 的键和查询，以及形状为 (batch_size, ..., seq_len, d_v) 的值；... 代表任意数量的其他批处理维度。输出

形状应为 (batch_size, ..., seq_len, d_v)。

还应支持用户可选提供的布尔掩码，形状为 (seq_len, seq_len)。掩码的行对应查询，列对应键：True 表示允许关注该键，False 表示不允许关注。对每个查询，所有掩码为 True 的位置上的注意力概率之和应为 1；掩码为 False 的位置上的概率应为 0。

原文说明，可在 softmax 之前，将掩码为 False 的位置对应的分数加上负无穷大，以实现屏蔽。

测试：实现 adapters.run_scaled_dot_product_attention。运行 uv run pytest -k test_scaled_dot_product_attention，测试三阶输入张量；运行 uv run pytest -k test_4d_scaled_dot_product_attention，测试四阶输入张量。

题目 multihead_self_attention：实现因果多头自注意力（5 分，原文第 25-26 页）

提交内容：将因果多头自注意力实现为 torch.nn.Module。构造函数至少应接收：

- d_model: int：Transformer 块输入的维度。
- num_heads: int：多头自注意力的头数。

令 h = num_heads，设置 d_k = d_v = d_model / h。原文给出的运算为：

MultiHead(Q, K, V) = Concat(head_1, ..., head_h)

head_i = Attention(Q_i, K_i, V_i)

MultiHeadSelfAttention(x) = W_O MultiHead(W_Q x, W_K x, W_V x)

其中，每个头使用 Q、K、V 特征维度上相应的切片；W_Q、W_K 的形状为 (h × d_k, d_model)，W_V 的形状为 (h × d_v, d_model)，W_O 的形状为 (d_model, h × d_v)。

实现正确后，应通过总共三次矩阵乘法分别计算键、值、查询投影。可选进阶目标：将三个投影合并为一个权重矩阵，使其只需要一次矩阵乘法。

因果掩码要求：位置 i 只允许关注 j ≤ i 的位置，禁止读取未来 token。应使用因果注意力掩码，复用前面缩放点积注意力对掩码的支持，而不是对每个前缀分别重新计算自注意力。可以使用 torch.triu 或广播后的索引比较来构造掩码。

RoPE 要求：只对查询和键应用 RoPE，不对值应用。把注意力头维度作为批处理维度；各头的查询和键使用完全相同的 RoPE 旋转规则。

测试：实现 adapters.run_multihead_self_attention，运行 uv run pytest -k test_multihead_self_attention。

题目 transformer_block：实现 Transformer 块（3 分，原文第 26 页）

实现原文第 3.4 节和图 2 所述的 pre-norm Transformer 块。每个块包含多头自注意力和 SwiGLU 前馈网络两个子层。每个子层都先执行 RMSNorm，再执行主要运算，最后加上残差连接。原文给出的第一个子层更新公式为：

y = x + MultiHeadSelfAttention(RMSNorm(x))

构造函数至少应接收：

- d_model: int：Transformer 块输入的维度。
- num_heads: int：多头自注意力的头数。
- d_ff: int：逐位置前馈网络内部层的维度。

测试：实现 adapters.run_transformer_block，运行 uv run pytest -k test_transformer_block。

提交内容：能够通过所提供测试的 Transformer 块代码。

题目 **transformer_lm**：实现 **Transformer** 语言模型（**3** 分，原文第 **26-27** 页）

按照原文第 3.1 节和图 1 组装完整模型：将 token 嵌入送入 `num_layers` 个 Transformer 块，再经过最终的归一化层和 LM head，得到词表上的未归一化分数 logits。

构造函数至少应接收前面 Transformer 块的所有构造参数，并额外接收：

- `vocab_size: int`：词表大小，用于确定 token 嵌入矩阵的维度。
- `context_length: int`：最大上下文长度，用于确定 RoPE 正弦、余弦缓冲区的维度。
- `num_layers: int`：Transformer 块的数量。

测试：实现 `adapters.run_transformer_lm`，运行 `uv run pytest -k test_transformer_lm`。

提交内容：能够通过上述测试的 Transformer 语言模型模块。

题目 **transformer_accounting**：**Transformer** 语言模型资源核算（**5** 分，原文第 **27-28** 页）

原文给出的核算规则：若矩阵 A 的形状为 `(m, n)`，B 的形状为 `(n, p)`，则矩阵乘法 AB 需要 `2mnp` 次浮点运算，即 `2mnp` FLOPs。先列出 Transformer 前向传播中的所有矩阵乘法，再换算为 FLOPs。

(a) 考虑使用本作业架构、大小与 GPT-2 XL 相当的模型，配置如下：

- `vocab_size = 50,257`
- `context_length = 1,024`
- `num_layers = 48`
- `d_model = 1,600`
- `num_heads = 25`
- `d_ff = 4,288`，即最接近 `(8/3) × 1,600` 的 64 的倍数。

如果按此配置构造模型，它有多少个可训练参数？假设每个参数都使用单精度浮点数表示，仅将模型加载到内存需要多少空间？

提交内容：用一至两句话回答。

(b) 指出完成这个 GPT-2 XL 规模模型的一次前向传播所需的矩阵乘法。这些矩阵乘法总共需要多少 FLOPs？假设输入序列包含 `context_length` 个 token。

提交内容：列出矩阵乘法及其说明，并给出所需 FLOPs 总数。

(c) 根据上面的分析，模型中哪些部分需要最多的 FLOPs？

提交内容：用一至两句话回答。

(d) 对以下模型重复分析：GPT-2 small（12 层，`d_model = 768`，12 个头）、GPT-2 medium（24 层，`d_model = 1024`，16 个头）、GPT-2 large（36 层，`d_model = 1280`，20 个头）。随着模型规模增大，Transformer 语言模型的哪些部分在总 FLOPs 中所占的比例会增加或减少？

提交内容：对每个模型，按组件列出 FLOPs 占一次前向传播总 FLOPs 的比例；另外用一至两句话说明模型规模变化如何影响各组件的 FLOPs 占比。

(e) 将 GPT-2 XL 的上下文长度增至 16,384。一次前向传播的总 FLOPs 如何变化？各组件的 FLOPs 相对占比如何变化？

提交内容：用一至两句话回答。

4 训练 Transformer 语言模型

本节需要实现交叉熵损失和 AdamW 优化器。训练循环还需要配套的数据加载、检查点保存和训练管理功能。

cross_entropy 实现交叉熵 1 分

原文第 28-29 页。

提交内容：编写一个函数，输入预测的 logits 向量 o_i 和目标 token x_(i+1)，计算交叉熵：

$\ell_i = -\log \mathrm{softmax}(o_i)[x_{i+1}]$。

实现必须满足以下要求：

- 为保证数值稳定性，减去最大的元素。
- 尽可能将 log 与 exp 相互抵消。
- 支持任意额外的批次维度，并返回整个批次上的平均损失。与第 3.2 节一样，批次类维度总在前面，词表维度在最后。

实现适配器 `adapters.run_cross_entropy`，运行 `uv run pytest -k test_cross_entropy` 验证。

评估时还需要报告困惑度（perplexity）。若长度为 m 的序列上，各位置的交叉熵为 $\ell_1,\ldots,\ell_m$，则：

$\mathrm{perplexity} = \exp((\ell_1 + \ldots + \ell_m) / m)$。

learning_rate_tuning 调整学习率 1 分

原文第 30-31 页。

使用原文第 30-31 页给出的 SGD 示例，将学习率分别改成 1e1、1e2、1e3，每次只训练 10 次迭代。观察每种学习率下的损失：下降得更快、更慢，还是发散，即随着训练进行而增大？

该原始示例使用随步数衰减的 SGD，更新式为 $\theta_{t+1} = \theta_t - \alpha / \sqrt{t+1} \cdot \nabla L(\theta_t; B_t)$。示例参数为一个以 5 倍标准正态随机数初始化的 10 × 10 权重矩阵，损失是所有权重平方的均值。请使用讲义中给出的示例代码完成本题。

提交内容：用 1-2 句话描述观察到的行为。

adamw 实现 AdamW 2 分

原文第 31-32 页。

提交内容：实现一个继承自 `torch.optim.Optimizer` 的 AdamW 优化器类。构造函数 `__init__` 应接收学习率 α，以及 β、ε、λ 等超参数。这里 β 包括 β_1 和 β_2，λ 是权重衰减系数。

原文第 29-30 页对优化器接口的要求如下：`__init__(self, params, ...)` 接收待优化的参数集合，或者具有不同超参数的参数组；将 `params` 和默认超参数字典传给父类构造函数。`step` 执行一次参数更新，在训练循环的反向传播之后调用，根据已有的 `p.grad` 原地修改参数；没有梯度的参数不进行更新。原文示例还接收可选的 `closure`，以兼容优化器 API，尽管本作业使用的优化器不需要它。

父类提供字典 `self.state`，将每个 `nn.Parameter` 对象映射到该参数的状态字典，可用于保存 AdamW 的一阶矩和二阶矩估计。

以下是原文算法 1 规定的更新关系，t 从 1 开始。初始化时 $m = 0$、$v = 0$，二者形状均与 θ 相同。每次采样批次 B_t，计算梯度 $g = \nabla_\theta \ell(\theta; B_t)$，然后按原文顺序更新：

$\alpha_t = \alpha \cdot \sqrt{1 - \beta_2^t} / (1 - \beta_1^t)$。

$\theta \leftarrow \theta - \alpha\lambda\theta$。

$m \leftarrow \beta_1 m + (1 - \beta_1)g$。

$v \leftarrow \beta_2 v + (1 - \beta_2)g^2$。

θ ← θ − α_t · m / (√v + ε)。

原文给出的常见取值包括 (β_1, β_2) = (0.9, 0.999)；LLaMA、GPT-3 等语言模型也采用 (0.9, 0.95)。ε 是保证数值稳定的小常数，例如 10^−8。

实现适配器 `adapters.get_adamw_cls`，确保通过 `uv run pytest -k test_adamw`。

adamw_accounting 计算 AdamW 训练的资源开销 2 分

原文第 32-33 页。假设所有张量均使用 `float32`。

(a) 运行 AdamW 时需要多少峰值内存？分别计算参数、激活、梯度和优化器状态的内存占用。用 `batch_size` 和模型超参数 `vocab_size`、`context_length`、`num_layers`、`d_model`、`num_heads` 表示结果，并假设 d_ff = (8/3) × d_model。

为简化计算，计算激活内存时只考虑以下组件：

- Transformer 块中的 RMSNorm。
- 多头自注意力子层：Q、K、V 投影，QK^T 矩阵乘法，softmax，对 value 的加权求和，以及输出投影。
- 逐位置前馈网络 SwiGLU：W_1、W_2、门控分支上的 SiLU、逐元素乘法，以及 W_3。
- 最后的 RMSNorm。
- 输出嵌入层。
- 对 logits 计算的交叉熵。

提交内容：参数、激活、梯度和优化器状态各自的代数表达式，以及总内存的表达式。

(b) 将上述结果代入 GPT-2 XL 规模的模型，得到一个仅依赖 `batch_size` 的表达式。在 80 GB 内存中，最大能使用多大的批次？

提交内容：形如 a × batch_size + b 的表达式，其中 a、b 为具体数值；以及最大批次大小。

(c) 一次 AdamW 更新需要多少 FLOPs？

提交内容：一个代数表达式及简短理由。

(d) 模型 FLOPs 利用率（MFU）衡量实际吞吐所对应的模型计算量相对于硬件理论峰值的利用程度。按题目给定条件，NVIDIA H100 对“float32”运算的理论峰值为 495 teraFLOP/s；原文说明这里实际指 TensorFloat-32，也称“bfloat19”。假设达到 50% MFU，在单张 H100 上以批次大小 1024 训练 GPT-2 XL 共 400K 步，需要多长时间？按原文引用的估算，反向传播的 FLOPs 是前向传播的两倍。

提交内容：所需训练小时数及简短理由。

learning_rate_schedule 实现带预热的余弦学习率调度 1 分

原文第 33-34 页。

编写函数，输入当前迭代 t、最大学习率 α_max、最小学习率 α_min、预热迭代数 T_w、余弦退火结束迭代 T_c，按以下规则返回 α_t：

- 预热阶段，t < T_w：α_t = (t / T_w) × α_max。
- 余弦退火阶段，T_w ≤ t ≤ T_c：α_t = α_min + 0.5 × [1 + cos(π × (t − T_w) / (T_c − T_w))] × (α_max − α_min)。
- 退火结束后，t > T_c：α_t = α_min。

实现适配器 `adapters.get_lr_cosine_schedule`，确保通过 `uv run pytest -k test_get_lr_cosine_schedule`。

gradient_clipping 实现梯度裁剪 1 分

原文第 34 页。

编写梯度裁剪函数，接收参数列表和允许的最大 L2 范数，并原地修改每个参数的梯度。使用 ε = 10^−6，与 PyTorch 默认值一致。

原文规定：将所有参数的梯度视为整体 g，计算其 L2 范数 ‖g‖_2。若范数小于上限 M，则保持不变；否则将梯度乘以 M / (‖g‖_2 + ε)。

实现适配器 `adapters.run_gradient_clipping`，确保通过 `uv run pytest -k test_gradient_clipping`。

5 训练循环

data_loading 实现数据加载 2 分

原文第 34-35 页。

提交内容：编写函数，接收以下输入：

- NumPy 数组 x：由 token ID 构成的整数数组。
- `batch_size`：批次大小。
- `context_length`：上下文长度。
- PyTorch 设备字符串，例如 `'cpu'` 或 `'cuda:0'`。

返回两个张量：采样得到的输入序列，以及对应的下一个 token 目标序列。两者形状都必须为 `(batch_size, context_length)`，内容均为 token ID，并放在指定设备上。

例如，当批次大小为 1、上下文长度为 3 时，`([x_2, x_3, x_4], [x_3, x_4, x_5])` 是一个可能的样本对。

实现适配器 `adapters.run_get_batch`，运行 `uv run pytest -k test_get_batch`。

大数据集要求：训练采样时，应使用内存映射方式加载数据，即 `np.memmap`，或者对用 `np.save` 保存的数据使用 `np.load(..., mmap_mode='r')`。指定与保存时一致的 `dtype`，并检查映射后的数据是否正确，例如 token ID 是否超出词表大小。

低资源提示：CPU 使用 `'cpu'`；Apple Silicon 使用 `'mps'`。模型与数据需要位于相同设备。

checkpointing 实现模型检查点 1 分

原文第 35-36 页。

实现以下两个函数，用于保存和加载检查点。检查点须包含恢复训练所需的模型状态、优化器状态和迭代编号。

`save_checkpoint(model, optimizer, iteration, out)`：将模型、优化器和迭代编号保存到 `out` 指定的位置。可以通过模型和优化器的 `state_dict` 方法取得状态，再使用 `torch.save(obj, out)` 保存。`out` 可以是路径或类文件对象；保存格式可自行选择，只要后续能够加载。

参数类型如下：

- `model: torch.nn.Module`
- `optimizer: torch.optim.Optimizer`
- `iteration: int`
- `out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`

`load_checkpoint(src, model, optimizer)`：从路径或类文件对象 `src` 加载检查点，恢复模型与优化器状态，并返回检查点中保存的迭代编号。可以使用 `torch.load(src)` 读取，再分别调用模型和优化器的 `load_state_dict`。

参数类型如下：

- `src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`
- `model: torch.nn.Module`
- `optimizer: torch.optim.Optimizer`

实现适配器 `adapters.run_save_checkpoint` 和 `adapters.run_load_checkpoint`，确保通过 `uv run pytest -k test_checkpointing`。

`training_together` 将各部分整合起来 4 分

原文第 36-37 页。

提交内容：编写训练脚本，运行训练循环，使用用户提供的输入训练模型。建议至少支持：

- 配置和控制模型与优化器的各项超参数。
- 使用 `np.memmap`，以节省内存的方式加载大型训练集和验证集。
- 将检查点序列化保存到用户指定的路径。
- 定期记录训练和验证表现，例如输出到终端，或使用 Weights and Biases 等外部服务。

原文建议让超参数便于配置，例如通过命令行参数传入，以支持后续重复运行不同实验。

6 生成文本

`decoding` 实现解码 3 分

原文第 37-38 页。

提交内容：实现从语言模型中解码并生成文本的函数。建议支持以下功能：

- 接收用户提供的提示词，即 token 前缀 x_1,...,x_t，持续采样补全文本，直到遇到 `<|endoftext|>`。
- 允许用户控制最多生成多少个 token。
- 根据用户指定的温度，在采样前对下一个 token 的分布应用 softmax 温度缩放。
- 根据用户指定的阈值进行 top-p 采样，也称核采样（nucleus sampling）。

原文规定的采样方式：取模型输出在最后一个位置的 logits 向量 v，用 softmax 得到下一个 token 的概率。每生成一个 token，就将其追加到输入中，继续生成，直到遇到结束 token 或达到生成长度上限。

温度缩放公式为：softmax(v, τ)_i = exp(v_i / τ) / Σ_j exp(v_j / τ)。

top-p 采样中，先把温度缩放后的概率 q 按从大到小排列，选取累计概率首次达到或超过 p 的最小 token 集合 V(p)。对集合内的概率重新归一化：token i 在集合内时，其概率为 q_i / Σ_(j∈V(p)) q_j；集合外概率为 0。

7 实验

experiment_log 实验记录 3 分

原文第 38-39 页。

为训练与评估代码建立实验跟踪功能，使其能够记录各次实验，以及损失随梯度更新步数和实际运行时间（wall-clock time）变化的曲线。定期评估验证损失，并同时记录步数和实际运行时间。

提交内容：实验记录功能的代码，以及本节后续各题的实验日志，即记录所有尝试的文档。

TinyStories 实验的基础配置

原文第 39-40 页。后续实验从以下配置出发：

- 词表大小 `vocab_size = 10000`。
- 上下文长度 `context_length = 256`。
- 模型维度 `d_model = 512`。
- 前馈层维度 `d_ff = 1344`，约为 (8/3) × d_model，同时是 64 的倍数。
- RoPE 参数 Θ = 10000。
- Transformer 层数为 4，注意力头数为 16。
- 总共处理约 327,680,000 个 token，即 `batch_size × total_step_count × context_length` 应约等于此数。

需要自行试验并选择合适的学习率、学习率预热设置、AdamW 的 β_1、β_2、ε，以及权重衰减。原文也鼓励改变词表大小和上下文长度，观察词表、模型行为、每步运行时间与最终困惑度的变化。

先取得训练好的 BPE 分词器，对训练集编码，再运行训练循环。原文预计，正确且高效的实现使用上述配置在单张 B200 上约需 20-30 分钟；若明显更慢，应检查数据加载、检查点保存、验证损失计算和批处理是否成为瓶颈。

原文调试建议包括：尝试过拟合一个小批次并使损失接近零；检查中间张量形状；监控激活、权重和梯度的范数。

learning_rate 调整学习率 3 分

原文第 40-41 页。预计实验资源：2 B200 小时。

以已经训练的基础模型为起点，回答以下问题。

(a) 对不同学习率进行超参数搜索，报告最终损失；若优化器发散，也要注明。

提交内容：多个学习率对应的学习曲线，并解释超参数搜索策略。

提交内容：一个在 TinyStories 验证集上逐 token 验证损失不超过 **1.45** 的模型。

(b) 常见经验认为，最佳学习率位于“稳定性的边缘”。研究使训练开始发散的学习率与最佳学习率之间的关系。

提交内容：逐渐提高学习率时的学习曲线，必须包含至少一次发散运行；分析其与收敛速度的关系。

低资源替代要求：若使用 CPU 或 MPS，将总处理 token 数减至约 40,000,000，并可将目标验证损失放宽到 **2.00**。原文示例使用 32 × 5000 × 256 = 40,960,000 个 token；在配有 36 GB 内存的 M4 Max 上，示例代码耗时约为 CPU 1 小时 22 分钟、MPS 36 分钟，5000 步时验证损失为 1.80。

原文另给出以下低资源提示：

- 若总训练步数为 N，建议让余弦学习率衰减在第 N 步恰好达到最小学习率。
- MPS 上不要像 CUDA 上那样启用 `torch.set_float32_matmul_precision('high')`。原文基于 PyTorch 2.9.0 的测试发现，这可能使用会导致训练不稳定的错误内核；这是讲义对应版本的提示。

- CPU 可尝试 `model = torch.compile(model)`；MPS 可尝试 `model = torch.compile(model, backend="aot_eager")` 以优化部分反向传播。原文说明 PyTorch 2.9.0 时 MPS 尚不支持 Inductor 编译。

`batch_size_experiment` 改变批次大小 **1** 分

原文第 41 页。预计实验资源：1 B200 小时。

将批次大小从 1 变化到 GPU 内存允许的上限。中间至少测试几种大小，包括 64 和 128 等常见取值。

提交内容：不同批次大小下的学习曲线。必要时重新优化学习率。

提交内容：用几句话讨论观察到的现象，以及批次大小对训练的影响。

`generate` 生成文本 **1** 分

原文第 42 页。

使用自己的解码器和训练好的检查点，报告模型生成的文本。为了得到流畅输出，可以调整温度、top-p 等解码参数。

提交内容：至少 256 个 token 的生成文本；若更早生成第一个 `<|endoftext|>`，则在该处停止。简要评价输出的流畅性，并讨论至少两个影响输出质量好坏的因素。

原文低资源提示说明，使用约 40M token 训练的模型仍应生成类似英语的文本，但流畅性可能低于完整训练配置。

`layer_norm_ablation` 去掉 **RMSNorm** 后训练 **1** 分

原文第 42 页。预计实验资源：0.5 B200 小时。

去掉 Transformer 中的全部 RMSNorm，然后训练。在之前的最佳学习率下会发生什么？降低学习率能否恢复稳定训练？

提交内容：移除 RMSNorm 后的学习曲线，以及采用该设置下最佳学习率的学习曲线。

提交内容：用几句话评价 RMSNorm 的影响。

`pre_norm_ablation` 实现后置归一化并训练 **1** 分

原文第 42-43 页。预计实验资源：0.5 B200 小时。

将前置归一化（pre-norm）的 Transformer 改为后置归一化（post-norm），训练并观察结果。

原文的 pre-norm 定义为：

z = x + MultiHeadSelfAttention(RMSNorm(x))。

y = z + FFN(RMSNorm(z))。

本题的 post-norm 定义为：

z = RMSNorm(x + MultiHeadSelfAttention(x))。

y = RMSNorm(z + FFN(z))。

提交内容：post-norm Transformer 的学习曲线，并与 pre-norm 的曲线比较。

`no_pos_emb` 实现 **NoPE** **1** 分

原文第 43 页。预计实验资源：0.5 B200 小时。

修改使用 RoPE 的 Transformer，完全移除位置嵌入信息，观察结果。这一变体称为 NoPE。

提交内容：比较 RoPE 与 NoPE 表现的学习曲线。

swiglu_ablation 比较 SwiGLU 和 SiLU 1 分

原文第 43-44 页。预计实验资源：0.5 B200 小时。

将 SwiGLU 前馈网络与使用 SiLU 激活、但没有门控线性单元的前馈网络进行比较。后者的定义为：

FFN_SiLU(x) = W_2 SiLU(W_1 x)。

SwiGLU 使用约 d_ff = (8/3) × d_model，并使 d_ff 为 64 的倍数。本题的 SiLU 基线应使用 d_ff = 4 × d_model，以便与具有三个权重矩阵的默认 SwiGLU 网络近似匹配参数量；SiLU 网络有两个权重矩阵。

提交内容：参数量近似相同的 SwiGLU 与 SiLU 前馈网络的学习曲线对比。

提交内容：用几句话讨论实验发现。

main_experiment 在 OpenWebText 上实验 2 分

原文第 44-45 页。预计实验资源：2 B200 小时。

在 OpenWebText 上训练语言模型，使用与 TinyStories 实验相同的模型架构和总训练迭代次数。观察模型表现。本实验可能需要重新调整学习率、批次大小等超参数。

提交内容：OpenWebText 上的学习曲线。描述其损失与 TinyStories 的差异，并解释应如何理解这些损失。

提交内容：OpenWebText 模型生成的文本，格式与 TinyStories 的生成结果要求相同。评价文本流畅性；解释为什么模型与计算预算相同，输出质量却更差。

低资源提示：对 GPU 资源有限的线上学习者，原文建议后续架构改动继续在 TinyStories 上测试，以验证损失衡量表现；在 OpenWebText 上把模型训练到能流畅生成文本需要更长时间。原文在此将参加课程排行榜表述为可选。

leaderboard 自行改进并参加排行榜 6 分

原文第 45-46 页。预计探索资源：10 B200 小时。

尝试改进 Transformer 架构或超参数，使语言模型在排行榜规则允许的计算量内取得尽可能低的验证损失。

排行榜规则：

- 一次提交在 B200 上最多运行 **45** 分钟，即 **0.75 B200** 小时。使用 SLURM 或 Modal 时，可在提交脚本中强制限制时间。
- 只能使用课程提供的 OpenWebText 训练集。
- 除上述两条外，原文没有列出其他排行榜限制。

原文建议先在 OpenWebText 的小子集或 TinyStories 上测试改动，再进行完整的 45 分钟训练。它提到可参考 Llama 3、Qwen 2.5 等开放模型，以及 NanoGPT speedrun 中的改动；例如共享输入与输出嵌入权重。若采用权重共享，可能需要减小嵌入层 / 语言模型输出头初始化的标准差。原文也提醒，小规模排行榜上有效的改动未必适用于更大规模预训练。

提交内容：最终记录的验证损失；以实际运行时间为横轴、明确显示训练时长少于 45 分钟的学习曲线；以及所做改动的说明。

排行榜提交预期至少优于验证损失 **5.0** 的朴素基线。

提交地址：`https://github.com/stanford-cs336/assignment1-basics-leaderboard`。